

INSTITUTO FEDERAL DE ALAGOAS
CAMPUS PALMEIRA DOS ÍNDIOS
CURSO TÉCNICO INTEGRADO AO ENSINO MÉDIO EM INFORMÁTICA

EDUARDO JOAQUIM SANTOS DA SILVA
GABRIEL BARBOSA

PALMEIRA DOS ÍNDIOS,AL
2026

EDUARDO JOAQUIM SANTOS DA SILVA
GABRIEL BARBOSA

Trabalho elaborado na disciplina de
Programação Web.
para obtenção de nota bimestral.
Professora: Higor Daniel Costa Cabral.

PALMEIRA DOS ÍNDIOS,AL
2026

SUMÁRIO

1 INTRODUÇÃO..... 4
2 VALIDAÇÃO CLIENT-SIDE VERSUS VALIDAÇÃO SERVER-SIDE.....4
2.1 Validação no lado do cliente (Client-Side)..... 4
2.2 Validação no lado do servidor (Server-Side).....5
3 ATRIBUTOS DE VALIDAÇÃO NATIVA DO HTML5..... 5
4 ESTADOS VISUAIS COM CSS: :VALID E :INVALID.....6
5 CONSIDERAÇÕES FINAIS..... 6
REFERÊNCIAS.....8

1 INTRODUÇÃO

Quando um usuário preenche um formulário na internet, seja para se cadastrar em um site, mandar uma mensagem de contato ou fazer uma compra, é preciso ter certeza de que os dados digitados estão corretos antes de serem enviados. Esse processo de checagem é chamado de validação de dados. Ele pode acontecer em dois momentos diferentes: no navegador do próprio usuário (*Client-Side*) ou no servidor que recebe os dados (*Server-Side*). Este trabalho mostra a diferença entre essas duas formas de validação e explica os atributos de validação nativa do HTML5 e os estados visuais do CSS usados no formulário de Contato/Ouvidoria criado pela dupla no portal da atividade. A formatação deste documento segue as normas da ABNT para trabalhos acadêmicos (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011).

2 VALIDAÇÃO CLIENT-SIDE VERSUS VALIDAÇÃO SERVER-SIDE

2.1 Validação no lado do cliente (Client-Side)

A validação *Client-Side* acontece direto no navegador do usuário, antes de os dados saírem do computador ou celular em direção ao servidor. Ela pode ser feita de forma nativa, só com atributos do HTML (como required, pattern, minlength e type), ou usando JavaScript para criar regras mais complexas, como comparar dois campos de senha, por exemplo. A vantagem é que o retorno é imediato: como tudo roda no próprio navegador, o usuário não precisa esperar resposta do servidor para saber que errou algum campo, o que deixa o site mais rápido e agradável de usar (*User Experience* – UX).

Só que a validação *Client-Side* não pode ser considerada segura sozinha. Como o código roda no navegador do próprio usuário, dá para desativar o JavaScript, mexer no HTML pelas ferramentas de desenvolvedor do navegador ou até mandar os dados direto para o servidor sem passar pelo formulário, ignorando as regras que foram colocadas no *front-end*. Por isso, ela deve ser vista como uma camada de conveniência para o usuário, e nunca como a única proteção de um sistema.

2.2 Validação no lado do servidor (Server-Side)

A validação *Server-Side* acontece depois, quando os dados já chegaram ao *back-end* da aplicação, em uma linguagem como PHP, Python, Node.js ou Java. Nessa etapa o servidor confere de novo todos os campos recebidos: verifica o tipo de dado, o tamanho, o formato (como e-mail e telefone), consulta o banco de dados para ver se já existe algo duplicado e aplica regras que não fazem sentido no *front-end*, como checar se um cupom de desconto ainda vale. Como o servidor fica sob controle da equipe que desenvolveu o sistema, essa é a validação que realmente garante a segurança, já que nenhuma informação vinda do usuário deve ser considerada confiável sem checagem.

O problema da validação *Server-Side* sozinha é que ela deixa a experiência do usuário mais lenta: como é preciso mandar os dados e esperar a resposta do servidor, o aviso de erro demora mais para aparecer, principalmente em uma internet mais lenta. Por isso, o mais comum na prática é usar as duas juntas: a validação *Client-Side* cuida da parte de usabilidade, dando respostas rápidas e evitando envios com erros óbvios, enquanto a validação *Server-Side* garante a segurança e a integridade final dos dados, sendo a barreira mais importante do sistema. O Quadro 1 resume as principais diferenças entre as duas abordagens.

Quadro 1 – Comparativo entre validação Client-Side e Server-Side

Critério	Client-Side	Server-Side
Onde executa	No navegador do usuário	No servidor da aplicação
Velocidade de resposta	Imediata	Depende da requisição
Nível de segurança	Baixo (pode ser burlada)	Alto (confiável)
Tecnologias comuns	HTML5 e JavaScript	PHP, Python, Node.js, Java etc.
Papel na aplicação	Melhora a experiência (UX)	Garante a integridade dos dados

Fonte: elaborado pelos autores (2026).

3 ATRIBUTOS DE VALIDAÇÃO NATIVA DO HTML5

O HTML5 trouxe um grupo de atributos que fazem a validação *Client-Side* sem precisar escrever nenhuma linha de JavaScript. Esses atributos são lidos direto pelo navegador, que impede o envio do formulário e mostra uma mensagem de erro padrão quando alguma regra não é cumprida. Segundo a documentação do MDN Web Docs (MOZILLA FOUNDATION, 2024a) e a especificação HTML Living Standard (WHATWG, 2024), os principais atributos usados no formulário de

Contato/Ouvidoria do portal foram:

a) **required** – obriga o preenchimento do campo antes de enviar o formulário. Se o campo estiver vazio, o navegador não deixa enviar e destaca o campo como inválido;

b) **pattern** – recebe uma Expressão Regular (Regex) que define o formato certo que o valor digitado precisa seguir. No campo de telefone, o pattern pede um formato parecido com (DD) 9XXXX-XXXX, misturando dígitos, parênteses e traço, que são opcionais;

c) **minlength e maxlength** – definem a quantidade mínima e máxima de caracteres aceitos em um campo de texto. O campo de mensagem usa minlength="15" para garantir que o usuário escreva algo com informação suficiente para a Ouvidoria.

4 ESTADOS VISUAIS COM CSS: :VALID E :INVALID

Segundo a documentação do MDN Web Docs sobre pseudo-classes CSS (MOZILLA FOUNDATION, 2024b), além de bloquear o envio, o navegador também aplica sozinho as pseudo-classes CSS nos campos do formulário, de acordo com o resultado da validação nativa. A pseudo-classe :invalid é aplicada em qualquer campo que não siga as regras definidas (como um required vazio ou um valor que não bate com o pattern), enquanto a pseudo-classe :valid é aplicada quando o conteúdo do campo está certo. Com isso dá para colorir bordas e fundo dos campos, dando um retorno visual imediato para o usuário, sem precisar de nenhuma lógica extra em JavaScript.

Um detalhe importante é que, por padrão, os campos required já aparecem como :invalid assim que a página carrega, mesmo antes de o usuário digitar qualquer coisa, o que deixa a tela com cara de erro logo no início. Para evitar isso, o formulário do portal usa a pseudo-classe :invalid junto com o seletor :not(:placeholder-shown), que só fica verdadeiro depois que o campo para de mostrar o texto de placeholder, ou seja, depois que o usuário começa a digitar. Assim, os campos só ficam vermelhos ou verdes depois que o preenchimento começa, o que deixa a experiência mais agradável.

5 CONSIDERAÇÕES FINAIS

Com essa pesquisa, deu para perceber que a validação *Client-Side* e a *Server-Side* não competem entre si, elas se completam. A validação nativa do HTML5, junto com os estados visuais do CSS (:valid e :invalid), dá uma resposta rápida e simples para o usuário, evitando erros bobos e melhorando a experiência de uso. Mesmo assim, em qualquer sistema real que dependa da segurança dos dados, é necessário repetir essas validações no servidor, já que só ele pode ser considerado confiável de verdade. O formulário de Contato/Ouvidoria feito nesta atividade usa os atributos required, pattern e minlength, além dos estados visuais estudados, sendo um exemplo prático de validação Client-Side aplicada em um caso real.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

MOZILLA FOUNDATION. **MDN Web Docs**: client-side form validation. Mountain View: Mozilla, 2024a. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn/Forms/Form_validation. Acesso em: 23 jul. 2026.

MOZILLA FOUNDATION. **MDN Web Docs**: :valid e :invalid — pseudo-classes CSS. Mountain View: Mozilla, 2024b. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/CSS/:valid. Acesso em: 23 jul. 2026.

WHATWG. **HTML Living Standard**: forms. [S. l.]: WHATWG, 2024. Disponível em: https://html.spec.whatwg.org/multipage/forms.html. Acesso em: 23 jul. 2026.